

Student: Nolan Jones

R#: R11789850

Professor: Eric Reec

Course: CS-4352-001

Date: May 5th 2026

Final Project Report

Scheduling Strategy:

The *If I fits I sits* algorithm is the name of the algorithm which I deployed this is an algorithm I designed myself but I'm sure it has probably been discovered before. The *If I fits I sits* algorithm operates by first checking if the range of movement of a person is encompassed by the range of movement of the elevator and the ,elevator is not full or has space, and is not broken or errored out, if the elevator doesn't match these criteria I check the next elevator, if all elevators are unavailable I then ask the server for the status of each elevators, upon insertion of a person into an elevator the "remainingCapacity" decrements and the "passengerCount" increases. It should be noted that the status of the elevators is only requested upon the failure of a person to be placed in an elevator meaning for 90% of the runtime there is desync between client-side elevator data and server-side elevator data. Also, the next person will first try to be placed on the next elevator after the last persons placement, this algorithm achieves a runtime of 1222 time steps. Based upon the project instructions students(me) are allotted a buffer time of .1 past the Reec Runtime Standard (RRS), the grading script claims any number ≤ 1401 is a 100, dividing this number by 1.1 gives us the Reec Runtime Standard (RRS) equivalent to 1273.6 - time steps. As such this algorithm seems to perform well above baseline on this assignment

despite it probably being a fortunate fluke. The justification for this algorithm is that it is simple, it reduces overhead by only guaranteeing bare minimum compliance and prioritizes reduction in ride time for long distance rides in order to hopefully free up capacity sooner. If the stupidity in the simplicity of this approach is not enough to justify it then I would have to say because it performs with a runtime this is 0.9594575303% of the Reec Runtime Standard (RRS) that's a whole 4% decrease in runtime.

Design Decisions:

Throughout the course of this several design decisions were made, the first being that C was my choice of language, and nearly all code is spliced from assignment 3, 4, and 5. Specifically API functions “init_put”, “init_get”, and “WriteCallback” were derived from assignment 3. The queue insertion and node creation were derived from project 5 you'll even note that the queue insertion check is a separate function just like assignment 5 which was left to emphasize congruency and similarity however, I opted not to pass it as a function parameter simply because it was unnecessary in this case. And lastly the threading style from assignment 4 was used mainly just that I used “pthread_mutex_t” and “pthread_cond_t” rather than manually implementing mutexes because it simplifies readability and writability compared to manual implementation.

Several other choices which were just resultant of the general project structure were: creation of a concatenate method to build strings, removal of header appending and x-session keys, and the requirement to find elevators based on their “bayId” rather than passing the index; which was an artifact of manual tests early on and simply hasn't been changed due to a lack of necessity. One peculiarity is my queue insertion algorithm which actually places individuals by longest distance to travel first, I employed this method to try to dispatch longer travelers earlier

hopefully decrease their time waiting for an elevator, which should also decrease their total time waiting to get to their destination. Others are the useless queue tails which I was originally going to use to simplify original queue node placement at the end of the queue without any checks however my original method for this caused a “zsh” error or something and I just reverted to the code that worked. Overall, this queue algorithm is only really employed when my code is repeatedly requesting elevator status’s because an individual couldn’t be placed and as such, although it could technically occur if request time takes too long. “nextInput()” could receive multiple others and place them in the queue, technically it could maybe also happen when calculating. The threading portion of this code met specifications but the most notable decision which I think was wise was only locking around queue pops and inserts, and on the compute thread I allowed the actual computation to happen after releasing “mutex1” and before grabbing “mutex2”, reducing probability of bottle necks when running the assignment algorithm.

One of the things I chose to do which eased development was I originally ran my algorithm in single threaded with only one queue. This make it much easier to verify if I could successfully complete the simulation with the algorithm. I then split the algorithm into the three threads. The first thread ‘request’ is responsible for calling “nextInput()” and placing people into a queue “personQueue” for the next thread “compute”. Compute then pops the first node only if the head does not equal NULL by using “pthread_cont_t” and computes the “best” placement, this information is updated on the node and the node is then passed to the next queue “dispatcherQueue”. The “dispatcher” thread is responsible for popping the first node in the previously mentioned queue again only when the head isn’t NULL and then dispatches the person to the elevator and frees memory.

Testing and Validation:

All testing and validation were first done locally utilizing the provided code from the GitHub repository, I used incremental testing to ensure individual methods could connect to endpoints send and receive the correct data then corrected the dispatch algorithm to properly ensure placement is valid rather than rely on server errors, the primary issue I ran into locally which didn't occur on the HPCC was my local instance of the server couldn't handle requests being spammed and my code would eventually error out without a slowdown in "elevatorStatus" and a slowdown in "nextInput()" when it was NONE, deterministic behavior was achieved through proper mutex locking and conditional variable use, along with short slowdown to prevent whenever the code would've accidentally spammed the server with packets. I am not actually certain that it is deterministic as I think there may be some wiggle room for dispatch ordering to change if the server or device request rate is slow enough but I'm about 70% certain that my usleep(200000) calls are preventing differenced in clock speeds from really affecting this. Mutex locks are only really acquired when accessing queues, allowing the compute algorithm to run while I get the next input and dispatch the previous. The most major bug I experienced was my zsh error that would cause my own code to fail when spamming the server with too many requests and that was fixed by adding a "usleep(200000)" to both the "request" thread when the user returned is "NONE" and to the "compute" thread after elevator statuses are retrieved.

Performance considerations:

The primary performance considerations are to do with spamming the server too much causing timeouts, I also do a lot of heap allocation with "calloc()" for strings for transferring data, I could maybe switch my "personId" and "bayId" to be set length arrays instead and simply read data into those locations instead of using temp variables then transferring. Overall, I think

that allocation of structure nodes for queues is a better option than an array-based solution for readability and simplicity. There should also be a better algorithm for dispatching however due to how well my *If I fits I sits* algorithm works it seems unnecessary. Timesteps are attempted to be minimized through reduction in wait time for further distance travelers and through an efficient placement algorithm that just wants to verify the route you take can get you to your destination.

Limitations and potential Improvements:

The primary improvement would be a proper dispatching algorithm as this could very well force the system to throttle horribly if experiencing any change in “nextInput()” ordering. It would also be nice to reduce the number of heap memory allocations I just like dynamic memory more. I should remove the elevator search as its excessive/unnecessary and remove the queue tail variables which are not used. One thing I was thinking about but never actually tried to implement was a request velocity limiter in the style of TCP RENO. It would operate differently obviously as it would slow down “elevatorStatus()” requests by maybe checking after 2 then 4 then 8 timesteps if they’re back-to-back due to elevators being full. There may be other small performance improvement in the code itself or better writability mechanisms but overall, I’m happy with the performance and organization of my code.

Based upon the wording in the assignment instructions it appears this is only a requirement of group submissions, as the individual distinctions are under that section. I certify that the submitted code was written by myself or collaboratively with my group members as

described in this document. I did not use AI-generated code or unauthorized external sources in the development of this project. I understand that the use of such materials constitutes academic dishonesty and will result in failure of the course.